

Reinforcement Learning of Optimal Portfolio Policies: Risk Aversion vs. Direct Return Optimization

Alexey Vaysburd

alexvays@stanford.edu

Extended Abstract

Motivation Financial portfolio optimization presents a challenging modeling problem, since rewards (portfolio returns) are stochastic and follow ever-evolving non-stationary patterns. On the other hand, while the agent’s actions affect the returns, they do not affect the environment (under assumption of minimal or no market impact). This combination makes offline training of a portfolio optimization policy with RL a feasible and interesting direction for exploration. The stochastic non-stationary setting with unreliable patterns of returns motivates the use of stochastic risk-sensitive policies and distributional predictions. My goal in this project was to evaluate and compare two reinforcement learning approaches for optimizing portfolio policy under uncertainty: one based on learning a risk aversion policy, and one based on direct optimization of portfolio weights.

Method I have evaluated two model architectures: (1) a risk-sensitive model that learns the distribution of predicted returns jointly with a stochastic risk aversion policy, and derives portfolio weights via the mean-variance optimization formula; and (2) a model that directly learns a stochastic portfolio weight policy via optimization of differentiable rewards. Both models use variational inference to encode latent representations of current market patterns and are trained using a 20-day rolling window replay buffer of market-hedged intraday returns. The framing of portfolio optimization in terms of a stochastic policy learned via RL represents the novel aspect of my approach.

Implementation Both model variants are implemented as neural networks with separate output heads for predicted means and log standard deviations of returns and policy parameters. The risk aversion model samples a log-normal λ (the risk aversion parameter) from a learned policy distribution, while the direct-optimization model samples weights directly from the learned policy (a Gaussian distribution) and applies a tanh transform. Both models are regularized using a KL loss on the latent variables and an L_2 loss for the zero prior on portfolio weights. The data consists of 1-minute intraday closes for over 4,000 U.S. stocks from 2020 to 2023. Model inputs include a vector of intraday returns at one-minute intervals between market open and the current time t_{cur} , where t_{cur} ranges between 2pm and 3:45pm at 15-minute increments. Other inputs include an embedding of the stock symbol and an embedding of the current time interval (t_{cur}). The training label is the cumulative return between t_{cur} and market close. The models’ performance is evaluated on rolling daily data prior to training on that data.

Results Both model architectures have produced positive cumulative returns. However, the direct portfolio weight policy has outperformed the risk aversion policy model in cumulative return and maximum drawdown (peak-to-trough negative cumulative return) metrics. Variational inference did not produce substantial performance improvements (in terms of cumulative rewards) but did help to a modest extent to make the model’s performance more consistent across the evaluation time range. Model performance proved to be significantly better when using market-hedged (vs. raw) returns.

Discussion The direct portfolio weight optimization approach turned out to be more effective, likely due to fewer intermediate transformations and a simpler mapping of rewards to the model’s parameters. Directions for future exploration include joint optimization of stocks’ weights across the portfolio instead of optimizing each stock individually, inclusion of additional input features, and accounting for transaction costs. The addition of prioritized experience replay (to include training examples representing the model’s biggest losses) may also help improve performance.

Conclusion Based on my evaluation results, direct optimization of a stochastic portfolio weight policy via differentiable rewards is an effective reinforcement learning model architecture for intraday portfolio construction. Learning risk aversion as a stochastic policy is a conceptually attractive approach from the perspective of its alignment with the common mean-variance portfolio optimization paradigm, but it introduces additional complexity in the model’s architecture without empirical benefit. This project’s results shows that RL-based portfolio optimization is a feasible and practical approach, as long as the model is carefully structured and tuned, and trained on market-hedged returns.

Reinforcement Learning of Optimal Portfolio Policies: Risk Aversion vs. Direct Return Optimization

Alexey Vaysburd
alexvays@stanford.edu

Abstract

This project explores reinforcement learning approaches for intraday portfolio optimization in a stochastic non-stationary financial environment where the agent's actions influence rewards but not the environment. The framing of portfolio optimization in terms of a stochastic policy learned via RL represents the novel aspect of my approach. I evaluated two model architectures: (1) a risk-sensitive model that predicts the distribution of returns and learns a stochastic policy over the risk aversion parameter, using the mean-variance utility formula to derive portfolio weights; and (2) a direct portfolio weight optimization model that learns a stochastic policy over portfolio weights by maximizing the expected return as a differentiable reward. Both models incorporate variational inference to capture uncertainty and to generalize across market regimes, using latent variables to represent market conditions. Training data consists of intraday returns for over 4,000 U.S. stocks from 2020–2023. The models' performance is evaluated on rolling daily data prior to training on that data. Evaluation results show that both model architectures produce positive cumulative returns, but direct portfolio weight optimization outperforms the risk aversion policy model. Model performance when trained and evaluated using market-hedged returns proved to be significantly better compared to using raw returns. Another finding is that variational inference improved model performance in terms of stability (reducing drawdowns), however it did not improve cumulative returns. This project confirms that it is feasible to use RL-based policies for decision making in the setting of financial markets with noisy non-stationary patterns of rewards.

1 Introduction

1.1 Problem Background

The project explores a Bayesian distributional approach to optimal sequential decision making (optimization of a financial portfolio) in a stochastic non-stationary environment with noisy rewards, in a setting where the agent's actions do not affect the environment itself (or do not affect the observations) but do affect the rewards.

As the specific problem setting, the project applies reinforcement learning to learn an optimal policy for intraday portfolio construction in the financial market. The environment is non-stationary due to the evolving market regimes, volatility shifts, and several type of seasonality (day-of-week, intraday, holiday, etc.) The problem is particularly challenging since the model has to be able to generalize effectively to environments with new patterns of returns, different from those on which it has trained.

Besides the portfolio optimization in the financial market, some other interesting and practically important examples of sequential-action optimization problems under uncertainty in a stochastic non-stationary environment unaffected by the model's actions include (1) driving route optimization based on real-time traffic conditions, and (2) flight path optimization based on weather conditions.

1.2 Objectives and Research Questions

The main goal the project has been to evaluate and compare performance of two novel model architectures using reinforcement learning to learn a portfolio optimization policy: (1) learning stochastic risk aversion policy jointly with a distribution of predicted returns (as a Bayesian estimator), and (2) learning portfolio weight policy via direct optimization of differentiable rewards.

Within this scope of the project, some specific research questions that I have explored include the following:

1. Which of the two model architectures results in better performance - learning the optimal risk aversion policy or learning portfolio weight policy?
2. What is the optimal construction of the neural network for this problem setting: As a plain stack of linear layers with ReLU activation or as an encoder / decoder architecture with a latent-variables representation and variational inference?
3. Does the model achieve better performance when training on market-hedged vs. raw (unhedged) returns?

2 Related Work

While sequential decision making in a stochastic environment is an important category of problems in reinforcement learning, the special case of optimal decision making under uncertainty in stochastic non-stationary environments unaffected by the actor's actions appears to be underexplored in Bayesian optimization and reinforcement learning literature.

Zha et al. (2021) propose Self-Supervised Reinforcement Learning algorithm that optimizes policies with supervised losses. Their algorithm generates multiple trajectories, selects trajectories with high rewards, and applies L2 loss to train the model's policy using the actions from high-reward trajectories as training labels. The paper highlights the link between a model's own output and its subsequent training data, but it doesn't discuss the steps to transform a deterministic supervised learning model into a stochastic-policy reinforcement learning model.

Eysenbach et al. (2020) discuss how to bridge supervised learning with reinforcement learning. They suggest that RL solves a sequence of supervised learning problems but uses optimized data with improved training labels. The paper compares different views on reinforcement learning, from the optimization perspective (optimizing policy), dynamic programming perspective (optimizing Q-function), and supervised learning perspective (optimizing policy together with the data distribution). However, the paper does not propose practical steps for transitioning from a supervised learning model to an RL model.

Bellemare et al. (2017) discuss the benefits of learning the value function as a predicted distribution (of the sum of trajectory rewards) rather than a predicted scalar value. This can be viewed as being complementary to the model's stochastic policy having the form of a distribution of actions rather than a single predicted "best" action as a scalar. Dabney et al. (2018) expands on these results and proposes using quantile regression to learn the empirical shape of the return distribution, which has the benefit of not having to make any particular assumptions on the parametric form of the distribution. Their approach aligns with my proposal to have the model predict distributions of returns at different time steps, which (given a particular portfolio composition) can then be translated into a trajectory of rewards. The limitation of these papers is that they do not discuss how to apply distributional predicted values within a stochastic portfolio optimization model.

Théate and Ernst (2022) discuss how to use the return probability distribution to learn sequential decision-making policies that are sensitive to risk, and to optimize the trade-off between risk minimization and expected return maximization. The paper proposes using a risk-based utility function in place of a standard Q-function. The paper mentions mean-variance optimization as one of the ways to connect distribution of returns with the optimal policy, but it does not present a specific model architecture that would combine learning the distribution of returns with learning risk aversion as a policy for optimal composition of the portfolio. This paper also doesn't cover the topic of transitioning from a deterministic supervised learning model to a stochastic-policy RL model.

Lim and Malik (2024) proposes using distributional reinforcement learning to train a risk-sensitive policy based on the CVaR risk metric (Conditional Value-at-Risk). They propose a risk-based utility function that combines a predicted distribution of returns with a risk metric (CVaR) computed from the distribution of returns. The novelty of my approach compared to this paper is that I propose learning the risk aversion parameter itself as a stochastic policy, beyond simply adjusting the policy by the CVaR of predicted returns.

3 Method

I have evaluated two model architectures: (1) risk-sensitive RL with Bayesian modeling of returns and risk aversion policy and (2) reinforcement learning of a stochastic portfolio weight policy via direct optimization of differentiable rewards.

3.1 Model architecture 1: Risk-sensitive RL with Bayesian modeling of returns and risk aversion policy

This model architecture comprises two neural networks, (1) a network that learns the distribution of predicted returns, and (2) a network that learns the risk aversion policy. The two networks share input features (including shared embeddings) but have separate layers. The detailed architecture is illustrated in Figure 1).

- The first network has two outputs $\mu_{ret,\theta}$ and $\log \sigma_{ret,\theta}$ specifying the distribution of predicted returns. This treats the predicted returns as a Bayesian estimator, i.e. a distribution rather than a scalar prediction.
- The model learns $\mu_{ret,\theta}$ and $\sigma_{ret,\theta}$ using NLL loss: $L_{NLL} = -\log p_{ret,\theta}(r|x)$, where $p_{ret,\theta}(y|x) \sim N(\mu_{ret,\theta}(x), \sigma_{ret,\theta}(x))$, with the returns $r_t(S)$ applied as training labels. I model predicted stock returns via a Gaussian distribution in the log space, which is aligned with empirical data and common industry practice.
- The second network outputs $\mu_{\lambda,\theta}$ specifying the mean of the lognormal policy distribution for the risk aversion parameter λ . The standard deviation of λ (in log space) is learned via a global parameter $\log \sigma_{\lambda,\theta}$.
- The model learns stochastic risk aversion policy $\lambda \sim \pi_{\theta}(\cdot|x)$ as a lognormal distribution: $\pi_{\theta}(\cdot|x) \sim \text{LogNormal}(\mu_{\lambda,\theta}(x), \sigma_{\lambda,\theta})$. I model λ via a lognormal distribution to ensure that the policy always outputs positive values of λ .
- I treat policy $\pi_{\theta}(\lambda|x)$ as a Bayesian estimator, sampling the risk aversion parameter λ from the learned distribution $\pi_{\theta}(\lambda|x)$ (with the reparameterization trick) at the training time, and applying expectation $\mathbb{E}_{\lambda \sim \pi_{\theta}(\cdot|x)}[\lambda]$ at the testing / evaluation time.
- The weight w of a stock in the portfolio is computed based on the mean-variance optimization formula, given the predicted distribution of returns $N(\mu, \sigma)$ and the risk aversion λ . Namely, w is computed as $w = \mu / (2\lambda\sigma^2)$. The model applies soft clipping to w via a tanh transformation as a way to improve stability and avoid outlier weights.
- The model's action λ is thus expressed in terms of a portfolio weight w for a given stock S . The reward for the model's action is the return for the corresponding portfolio (stock S with position weight w) between the current time t and End-of-Day: $R(S, t) = w \cdot r_t(S)$.
- For the purpose of computing the rewards, the return label for stock S as of time t is constructed as

$$r_t(S) = (\log P_{end_of_day}(S) - \log P_t(S)) - (\log P_{end_of_day}(\text{market}) - \log P_t(\text{market}))$$

in the market-hedged version of the model, and as

$$r_t(S) = (\log P_{end_of_day}(S) - \log P_t(S))$$

in the unhedged version.

- Since the reward is a differentiable function of w , and through w is a differentiable function of the risk aversion policy $\pi_{\theta}(\lambda|x)$, the model's architecture allows us to apply the negated reward $-R(S, t)$ as the loss, and to train the risk aversion policy $\pi_{\theta}(\cdot|x)$ to maximize the portfolio returns: $\mathcal{L} = -R(S, t) = -w \cdot r_t(S)$.

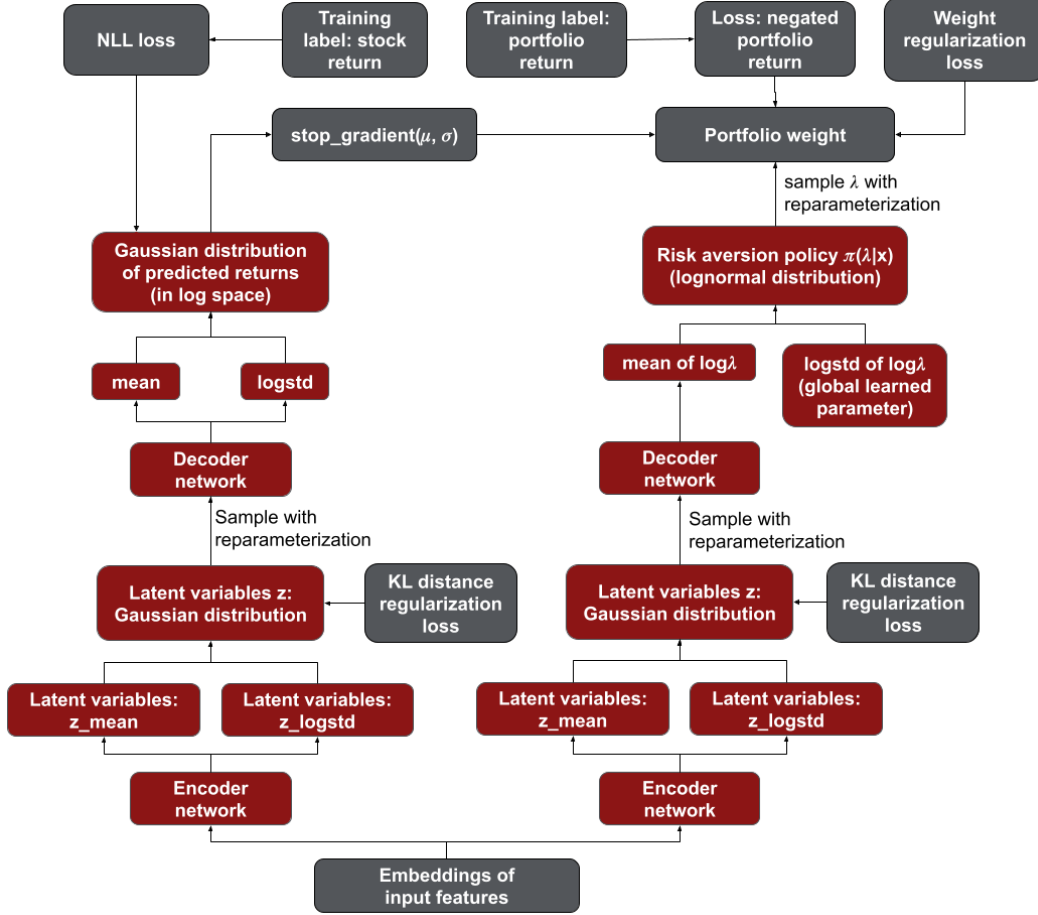


Figure 1: Model architecture learning risk aversion policy $\pi(\lambda | x)$

3.2 Model architecture 2: Learning a stochastic portfolio weight policy via direct optimization of differentiable rewards

With this architecture, the model comprises a single neural network that learns a stochastic portfolio weight policy trained to maximize the rewards. The detailed architecture is illustrated in Figure 2.

- The neural network outputs $\mu_{w,\theta}(x)$ which represents the mean of the portfolio weight policy distribution $\pi_\theta(\cdot|x)$.
- The standard deviation of the portfolio weight policy distribution is learned (in log space) via a global learned parameter $\log \sigma_{w,\theta}$.
- The weight w of a stock in the portfolio is constructed as $w = \tanh(z)$ where $z \sim \pi_\theta(\cdot|x) = N(\mu_{w,\theta}(x), \sigma_{w,\theta})$. At the training time, z is sampled from the policy distribution (with a reparameterization trick so as to enable backpropagation of gradients). At the evaluation time, z is set deterministically to $\mu_{w,\theta}(x)$. The tanh transformation is applied to improve model stability and to avoid outlier weights.
- The model applies the negated reward (portfolio return) as the loss, same as with the first architecture: $\mathcal{L} = -R(S, t) = -w \cdot r_t(S)$. Since the loss (the negated reward) is differentiable with respect to the model's policy $\pi_\theta(\cdot|x)$, the model is able to learn optimal policy directly by training to maximize the portfolio returns.

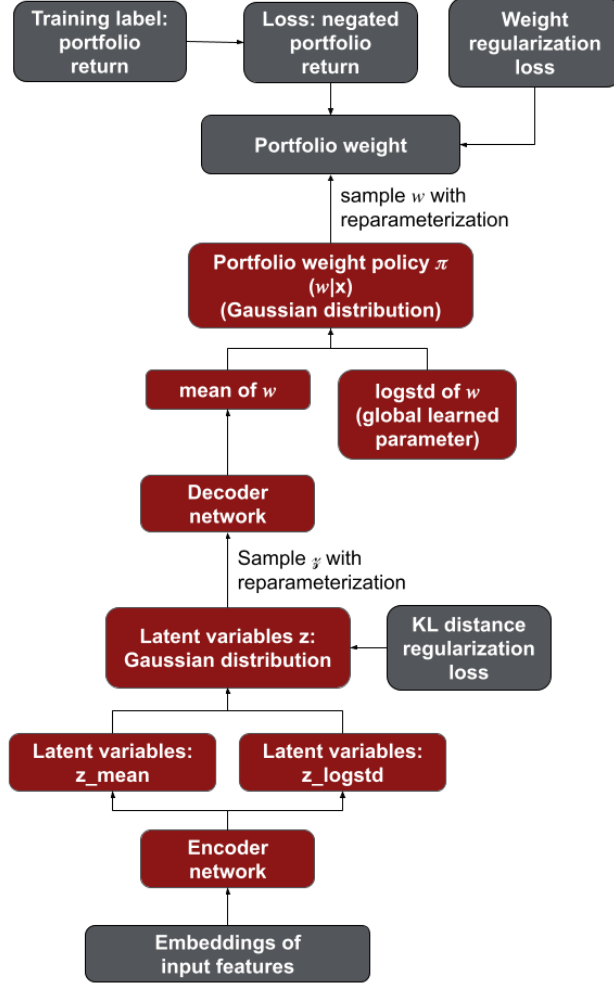


Figure 2: Model architecture learning portfolio weight policy $\pi(w | x)$

3.3 Variational inference

Both model architectures use a variational encoder architecture with 32-dimensional latent variables. This design is intended to improve generalization and make the model more resistant to overfitting, by learning uncertainty as part of the structure in the noisy market data. Variational inference can be viewed as a form of Meta-RL, where the latent variables represent the market environment on a given day as a type of the "task".

- The Encoder network outputs the tensor of distribution means μ_z (of width 32) and a tensor of standard deviations (in log space) $\log \sigma_z$, also of width 32, which jointly specify the distribution of the 32-dimensional latent variable z with independent dimensions (i.e with a diagonal covariance matrix).
- At the training time, z is sampled from the Encoder's distribution: $z \sim N(\mu_z, \sigma_z)$, using the reparameterization trick to enable backpropagation of gradients into μ_z and $\log \sigma_z$. At the evaluation time, z is set to the mean μ_z of the distribution.
- The sampled values of z are applied as inputs into the Decoder network.

3.4 Regularization losses

- Both model architectures apply L_2 regularization loss $\mathcal{L}_{reg} = \beta_{reg} \cdot w^2$ to penalize for non-zero position weights w . This is used to express a zero prior on the weights. The β_{reg} scaling factor is a hyperparameter (set to 1).
- A KL distance loss is applied to the distribution of latent variables output by the encoder to express a $N(0, 1)$ prior on the distribution of latent variables z . The KL distance loss has the form $\mathcal{L}_{KL} = -\beta_{KL} \cdot \frac{1}{2} (1 + 2 \log \sigma_z - \mu_z^2 - \sigma_z^2)$. The β_{KL} scaling factor is a hyperparameter (set to 1e-3).

4 Experimental Setup

4.1 Training Data

The models have been trained using intraday prices (1-minute bars) for over 4,000 US stocks obtained from Databento, spanning 1/2020 through 12/2023. Stocks with daily dollar volume below \$1 million are excluded.

4.2 Construction of Training Examples

A training example for stock S at time t comprises the history of intraday returns for S between the current day's market open and $t - 1$, either as raw returns (in the unhedged version) or adjusted by the market's returns (in the market-hedged version). The market is represented via SPY (S&P500 ETF) as a proxy. The values of t on each day range between 2pm and 3:45pm at 15-minute increments. Other model inputs include an embedding of the stock symbol and an embedding of the current time t .

The training labels are constructed using the returns between current time t and market close on the same day, either as raw returns (in the unhedged version) or adjusted by the market return (in the market-hedged version).

The returns are calculated as changes in prices in the log space, for example the raw (unhedged) return for stock S between time t and end of day is calculated as $R_t(S) = \log P_{end_of_day}(S) - \log P_t(S)$.

4.3 Training/Evaluation Phases and the Replay Buffer

The model advances in evaluation and training sequentially (forward in time) one day at a time, with the following procedure:

- When advancing to the next day T , the model is evaluated on that day's data first, before ever using data from T in training. During evaluation, the model's cumulative reward is updated with the mean reward in each evaluation batch.
- The training examples for T are then added to the 20-day rolling window replay buffer, with the data for $T - 20$ (the oldest retained day) pruned from the buffer (thus the replay buffer contains training examples from a 20-day rolling window). Training batches are constructed from randomly shuffled examples within the replay buffer.

See the diagram in Figure 3 for the illustration of the replay buffer and the evaluation/training steps performed for each training day T .

4.4 Performance Metrics

I evaluate and compare the models' performance via two main metrics, (1) the cumulative rewards and (2) the maximum drawdowns during the training/evaluation period 1/2020 - 12/2023. The drawdown is calculated as the maximum difference between a peak in cumulative rewards and a subsequent trough.

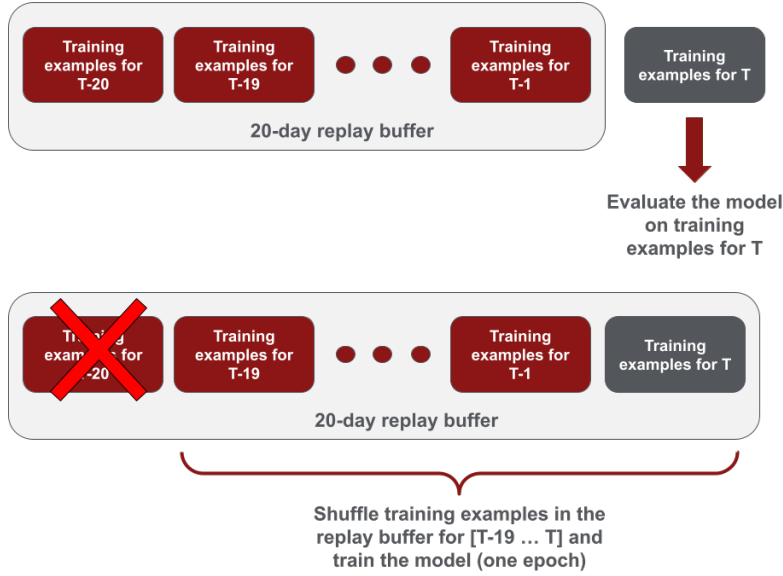


Figure 3: Replay buffer with sequential evaluation and training

5 Results

Both model architectures have resulted in positive returns. However, the cumulative returns have been higher when learning portfolio weight policy directly compared to the architecture that learns the distribution of predicted returns jointly with the risk aversion policy and derives portfolio weight via the mean-reversion optimization formula.

The direct portfolio weight optimization approach turned out to be more effective, likely due to fewer intermediate transformations and a simpler mapping of rewards to the model’s parameters.

Variational inference (having the decoder network output latent variables as a multi-dimensional distribution) has not made a significant impact on the model’s performance, compared to a version not using variational inference. It did help reduce drawdowns yet did not help increase cumulative returns. It is possible that the limited set of input features applied with the model was not expressive enough to fully realize the potential benefits from data compression, regularization, and explicit modeling of epistemic uncertainty via the latent variables.

It proved more tractable to design an RL model to learn a reasonably-performing portfolio construction policy using hedged (market-neutral) returns compared to using raw unhedged stock returns. This must be a reflection of higher noise-to-signal ratio and less-generalizable patterns with raw returns.

5.1 Quantitative Evaluation

Quantitative performance metrics are summarized in Table 5.1.

- The model configurations using raw unhedged returns had several-times-higher drawdowns in cumulative returns (1.4e4, 1.2e4) compared to the models trained on market-hedged returns (2.7e3, 3.6e3, 3.3e3).
- The smallest drawdown (2.7e3) was achieved with a model using direct reward optimization and variational inference, and trained on market-hedged returns.
- The highest cumulative reward (2.33e4) was achieved with a model using direct reward optimization and market-hedged returns, without variational inference. The higher cumulative reward was counterweighted by a relatively higher drawdown (3.3e3) compared to the model configuration with variational inference enabled.

Model version	Cumulative return (reward) at the end of the training/evaluation period	Maximum drawdown in cumulative returns (peak to trough)
Direct reward optimization using variational inference and market-hedged returns	1.8e4	2.7e3
Risk aversion policy optimization using variational inference and market-hedged returns	1.0e4	3.6e3
Direct reward optimization using variational inference and raw (unhedged) returns	1.26e4	1.4e4
Risk aversion policy optimization using variational inference and raw (unhedged) returns	1.23e4	1.2e4
Direct reward optimization with market-hedged returns, NOT using variational inference	2.33e4	3.3e3

Table 1: Performance comparison between different model architectures and hyperparameter settings

5.2 Qualitative Analysis

5.3 Model Performance With Risk Aversion Policy vs. Portfolio Weight Policy

The cumulative returns have been higher, and drawdowns lower, when learning portfolio weight policy directly (orange series in Figure 4) compared to the model architecture in which the distribution of predicted returns is learned jointly with the risk aversion policy and the portfolio weight is constructed via the mean-reversion optimization formula (blue series in Figure 4).

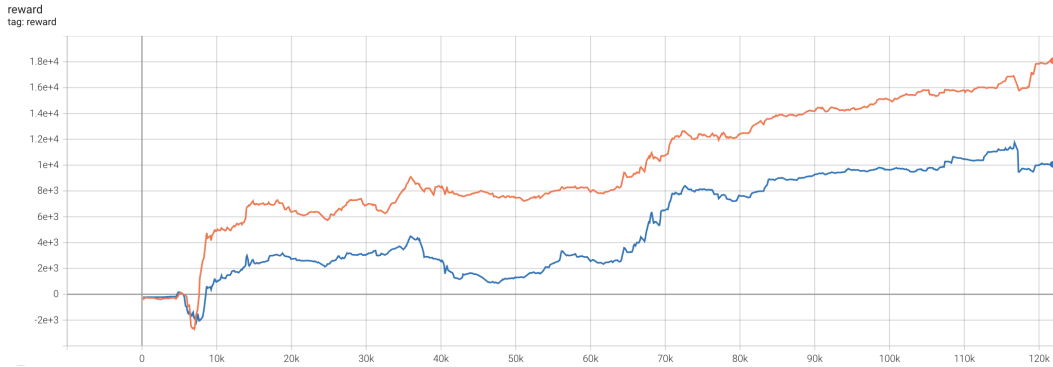


Figure 4: Cumulative returns with the model learning risk aversion policy (blue series) vs. learning portfolio weight policy via direct optimization of differentiable rewards (orange series).

5.4 Model Performance With Variational Inference Enabled vs. Disabled

Cumulative returns (but also drawdowns) have been higher in the model configuration with variational inference disabled (the blue series in Figure 5) compared to the model configuration with variational inference enabled (the orange series in Figure 5).

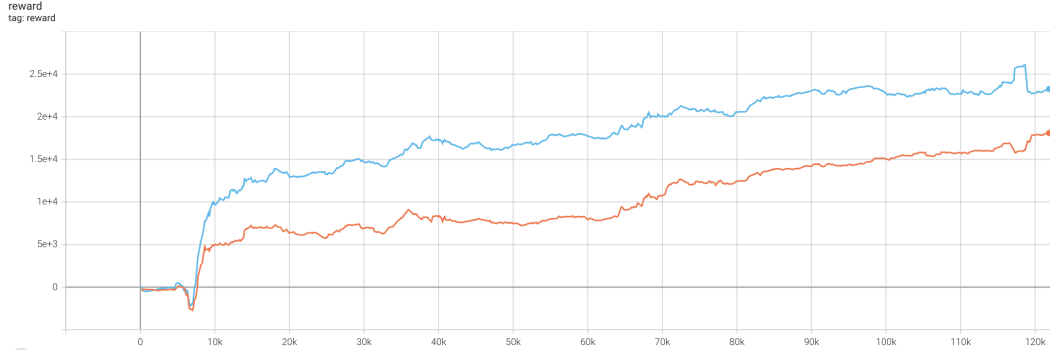


Figure 5: Cumulative returns for the model learning portfolio weight policy via direct optimization of differentiable rewards, with variational inference enabled (orange series) vs. disabled (blue series).

5.5 Model Performance Using Raw Unhedged Returns

Model performance when using raw unhedged returns (Figure 6) is inferior in terms of the cumulative return as well as the drawdown, both for the architecture learning risk aversion policy (red series) and the architecture learning portfolio weight policy via direct optimization of differentiable rewards (blue series).

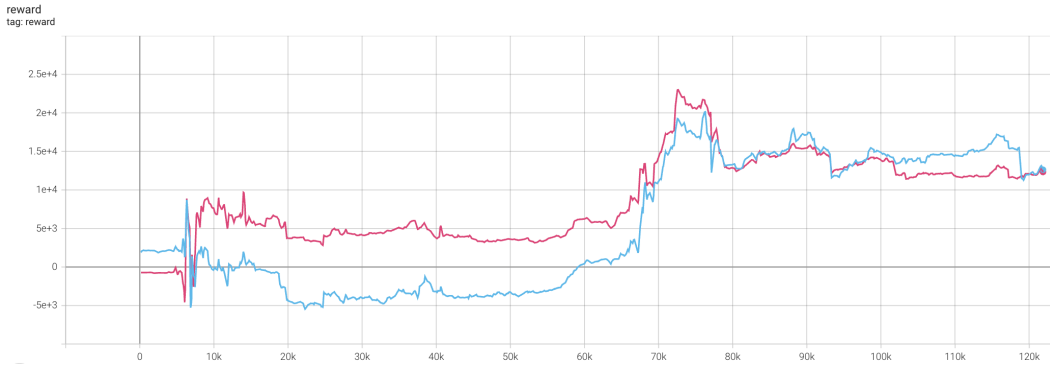


Figure 6: Model performance using raw unhedged returns, for the architecture learning risk aversion policy (red series) vs. the architecture learning portfolio weight policy via direct optimization of differentiable rewards (blue series). Performance is less consistent (higher volatility, higher and longer drawdowns) compared to the models trained and evaluated on market-hedged returns.

6 Discussion

The biggest challenge I encountered with this project was to get the model to produce a reasonably good time series of cumulative returns (with a mostly smooth upward slope and constrained drawdowns) for the first time. The main breakthrough occurred when I switched from using raw unhedged to market-hedged returns. It turned out to be much easier for the RL policy to generalize patterns of hedged stock returns than raw unhedged ones.

For simplicity and debuggability I designed the model to optimize the weight of each stock in the traded universe individually. A more practical direction for future work will be to optimize the weights of all stocks jointly, e.g. as a single multi-dimensional policy.

It may be helpful to combine the 20-day rolling window replay buffer with a prioritized experience replay (a secondary replay buffer) that would include training examples on which the model incurred its biggest losses, with no expiration.

For simplicity I used the bare minimum of input features in the models evaluated in this project, mainly the intraday returns preceding the "current time". For future work, a more expressive set of input features will help improve the model's performance.

In order for the portfolio optimization model to be practically useful, it will have to account for transaction costs and various portfolio composition constraints (e.g. liquidity constraints).

7 Conclusion

Based on my evaluation results, direct optimization of a stochastic portfolio weight policy via differentiable rewards is an effective reinforcement learning model architecture for intraday portfolio construction.

Learning risk aversion as a stochastic policy may appear to be a conceptually attractive approach from the perspective of its alignment with the common mean-variance portfolio optimization paradigm, but it introduces additional complexity in the model's architecture without empirical benefits.

This project's results show that RL-based portfolio optimization is a feasible and practical approach, as long as the model is carefully structured and tuned, and trained on market-hedged returns.

The future directions include expanding from a single position optimization to a joint multi-stock portfolio optimization, enriching the model's inputs with additional informative features, and exploring more advanced neural network architectures such as convolutional networks, LSTM, and Transformers / self-attention layers, which may be able to learn complex intraday patterns of sequential inputs more effectively than a simple dense network.

8 Team Contributions

Alexey Vaysburd: Project idea, model design, implementation, training/evaluation, iterations over different designs and model versions, writeup of the project proposal, milestone report, poster presentation, and final report.

References

- Marc G. Bellemare, Will Dabney, and Rémi Munos. 2017. A Distributional Perspective on Reinforcement Learning. *CoRR* abs/1707.06887 (2017). arXiv:1707.06887 <http://arxiv.org/abs/1707.06887>
- Will Dabney, Mark Rowland, Marc G. Bellemare, and Rémi Munos. 2018. Distributional Reinforcement Learning with Quantile Regression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- Ben Eysenbach, Aviral Kumar, and Abhishek Gupta. 2020. Reinforcement Learning is Supervised Learning on Optimized Data. <https://bair.berkeley.edu/blog/2020/10/13/supervised-rl/>. Berkeley AI Research Blog.
- Seong Hoon Lim and Imran Malik. 2024. Distributional Reinforcement Learning for Risk-Sensitive Policies. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=wSVEd3Ta42m> Available at: <https://openreview.net/forum?id=wSVEd3Ta42m>.
- Thomas Théate and Damien Ernst. 2022. Risk-Sensitive Policy with Distributional Reinforcement Learning. *arXiv preprint arXiv:2212.14743* (2022). arXiv:2212.14743 [cs.LG] <https://arxiv.org/abs/2212.14743>
- Daochen Zha, Kaixiong Zhou, Qiang Huang, Xia Hu, and Pengtao Xie. 2021. Simplifying Deep Reinforcement Learning via Self-Supervision. *arXiv preprint arXiv:2106.05526* (2021).